



BÀI 08: CÁC LỚP CƠ SỞ VÀ CÁC LỚP CẤU TRÚC DỮ LIỆU

- ❑ Cấu trúc mảng trong Java
- ❑ Một số lớp dữ liệu cơ bản trong Java
- ❑ Các lớp cấu trúc dữ liệu
- ❑ Các lớp tuyến tập (container classes)
- ❑ Bài tập



BÀI 08: CÁC LỚP CƠ SỞ VÀ CÁC LỚP CẤU TRÚC DỮ LIỆU

- ❑ **Cấu trúc mảng trong Java**
- ❑ Một số lớp dữ liệu cơ bản trong Java
- ❑ Các lớp cấu trúc dữ liệu
- ❑ Các lớp tuyến tập (container classes)
- ❑ Bài tập



Cấu trúc mảng trong Java

□ Định nghĩa:

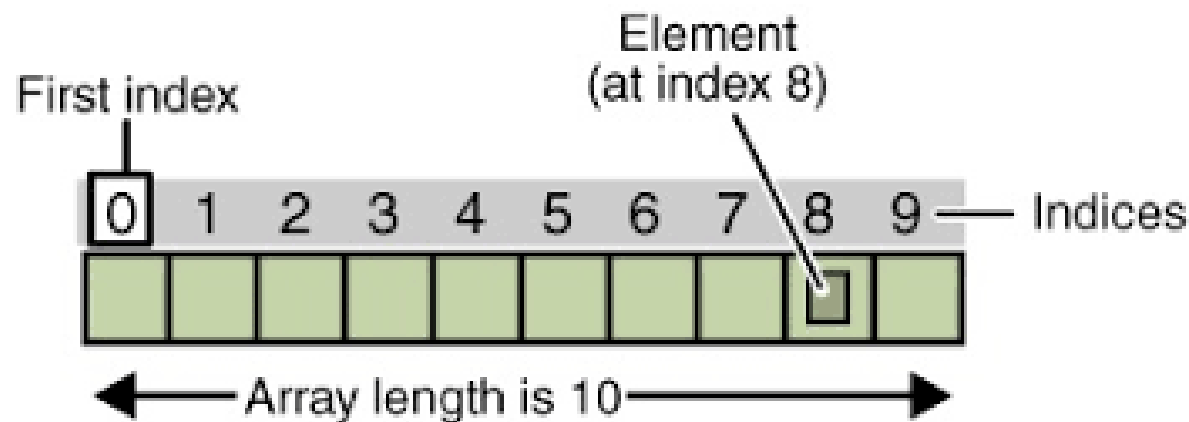
- Mảng là một cấu trúc dữ liệu lưu trữ một tập hợp các giá trị có cùng kiểu dữ liệu, được truy cập thông qua chỉ số (index).
- Kích thước của mảng là cố định sau khi được khởi tạo và không thể thay đổi trong quá trình thực thi chương trình.



Cấu trúc mảng trong Java

❑ Đặc điểm:

- ❑ Truy cập thông qua chỉ số
- ❑ Kiểu dữ liệu đồng nhất
- ❑ Được lưu trữ liên tiếp
- ❑ Kích thước cố định





Cấu trúc mảng trong Java

❑ Khai báo:

❑ Một chiều: <Kiểu DL>[] <Biến mảng>;
 <Kiểu DL> <Biến mảng>[] ;

❑ Hai chiều: <Kiểu DL>[][] <Biến mảng>;
 <Kiểu DL> <Biến mảng>[][] ;
 <Kiểu DL>[] <Biến mảng>[] ;



Cấu trúc mảng trong Java

❑ Ví dụ:

int[] D, E, F;

int A[], B[], t, s;

double X[][], Y[];

double[][] M, N;

→ A, B, D, E, F: Mảng 1 chiều các số nguyên;

→ t, s: Biến nguyên thông thường

→ X, M, N: Mảng 2 chiều các số thực;

→ Y Mảng 1 chiều các số thực



Cấu trúc mảng trong Java

❑ Tạo lập đối tượng mảng:

<Tên mảng> = new <Kiểu các phần tử >[<Số phần tử>];

A= **new int**[100];

=> A: Mảng chứa 100 số nguyên

<Tên mảng> = new <Kiểu các phần tử >[<Số hàng>][<Số cột>];

M=**new double**[3][4];

=> M Mảng 2 chiều chứa 3x4=12 số thực



Cấu trúc mảng trong Java

❑ Sử dụng mảng:

- ❑ Mảng được truy cập thông qua phần tử mảng.
- ❑ Mỗi phần tử mảng được truy cập thông qua tên mảng và chỉ số truy cập tương ứng.

❑ Cú pháp:

- ❑ Mảng 1 chiều: `<Tên mảng>[chỉ_số];`
- ❑ Mảng 2 chiều: `<Tên mảng>[Chỉ_số_hàng][Chỉ_số_cột];`
- ❑ VD: `k=A[10];` `A[9]=100;` `x=M[2][2];`
- ❑ Trong đó, chỉ số truy cập là một số nguyên không âm và không được vượt quá giới hạn tương ứng.
- ❑ Nếu vi phạm => Ng.Lệ: `ArrayIndexOutOfBoundsException`



Cấu trúc mảng trong Java

❑ Quy trình làm việc với MẢNG trong Java:

❑ B1: Khai báo cấu trúc mảng

- ❑ Mảng 1 chiều: **int** A[];
- ❑ Mảng 2 chiều: **double** M[][];

❑ B2: Nhập/xác định kích thước mảng: (**m, sh, sc**)

❑ B3: Tạo lập đối tượng mảng

- ❑ Mảng 1 chiều: A = **new int**[m];
- ❑ Mảng 2 chiều: M=**new double**[sh][sc];

❑ B4: Truy cập/sử dụng mảng: Thông qua tên mảng + Chỉ số

- ❑ Mảng 1 chiều: **x= A[k];** **A[i]=10;**
- ❑ Mảng 2 chiều: **y = M[i][j]** **M[i][j] = 100;**

**Trong đó: k, i, j là các số nguyên không âm; nhỏ hơn số chiều tương ứng.
Nếu vi phạm => Ngoại lệ: ArrayIndexOutOfBoundsException**



Cấu trúc mảng trong Java

❑ Ví dụ tổng hợp: Mảng các đối tượng

Giả sử thông tin về mỗi khách hàng gồm: Mã , họ tên, tuổi, địa chỉ liên hệ, số điện thoại, email.

1. *Hãy xây dựng lớp **KhachHang** có:*

- Các toán tử tạo lập
- Phương thức để nhập thông tin cho một khách hàng
- Phương thức để hiển thị thông tin của khách hàng hiện thời

2. *Xây dựng lớp **QLKhachHang** có cài đặt các phương thức thực hiện các công việc sau:*

- Nhập vào thông tin cho một danh sách gồm n khách hàng (n -nhập từ bàn phím)
- Sắp xếp danh sách khách hàng theo thứ tự giảm dần của tuổi
- Hiển thị ra màn hình thông tin của tất cả các khách hàng có tên là “Nguyen Van Duc” (không phân biệt chữ hoa hay chữ thường)



Cấu trúc mảng trong Java

❑ Ví dụ tổng hợp: Mảng các đối tượng

KhachHang
private String ma private String hoTen private int tuoi private String diaChi private String sdt private String email
KhachHang() KhachHang(...) public void nhapTT() public void hienThiTT() public int getTuoi() public String getTen()

QLKhachHang
private int n private KhachHang dskh[]
public void nhapDSKH() public void hienThiDSKH() public void sxTheoTuoi() public void timTheoTen() main(...)

HAS-A
relationship



Cấu trúc mảng trong Java

```
import java.io.*;
class KháchHang{
    // Khai báo các thuộc tính
    private String ma;
    private String hoTen;
    private int tuoi;
    private String diaChi;
    private int sdt;
    private String email;
    // Khai báo các toán tử tạo lập
    KháchHang(){
    }
    KháchHang(String m){ma=m;}
    KháchHang(String m, String ht, int t, String dc, int dt, String em){
    }
    // Khai báo các phương thức
    public int getTuoi() { return tuoi;}
    public String getHoTen() { return hoTen;}
    public void nhapTT() {}
    public void hienThiTT() {}
}
public class QLKhachHang {
    int n; // Số phần tử của danh sách
    KháchHang dskh[]; // Danh sách chứa các khách hàng
    public void nhapDSKH() {}
    public void hienThiDSKH() {}
    public void sxTheoTuoi() {}
    public void timTheoTen() {}
    public static void main(String arg[]) {}
}
```



BÀI 08: CÁC LỚP CƠ SỞ VÀ CÁC LỚP CẤU TRÚC DỮ LIỆU

- ❑ Cấu trúc mảng trong Java
- ❑ **Một số lớp dữ liệu cơ bản trong Java**
- ❑ Các lớp cấu trúc dữ liệu
- ❑ Các lớp tuyến tập (container classes)
- ❑ Bài tập



Một số lớp dữ liệu cơ bản trong Java

❑ **Khái niệm:**

- ❑ Các lớp dữ liệu cơ bản được gọi là wrapper classes (lớp bao), được sử dụng để bao bọc các kiểu dữ liệu nguyên thủy (primitive types).
- ❑ Mỗi kiểu dữ liệu nguyên thủy đều có một lớp bao bọc tương ứng, cung cấp các phương thức hữu ích và các tính năng bổ sung mà các kiểu dữ liệu nguyên thủy không có.



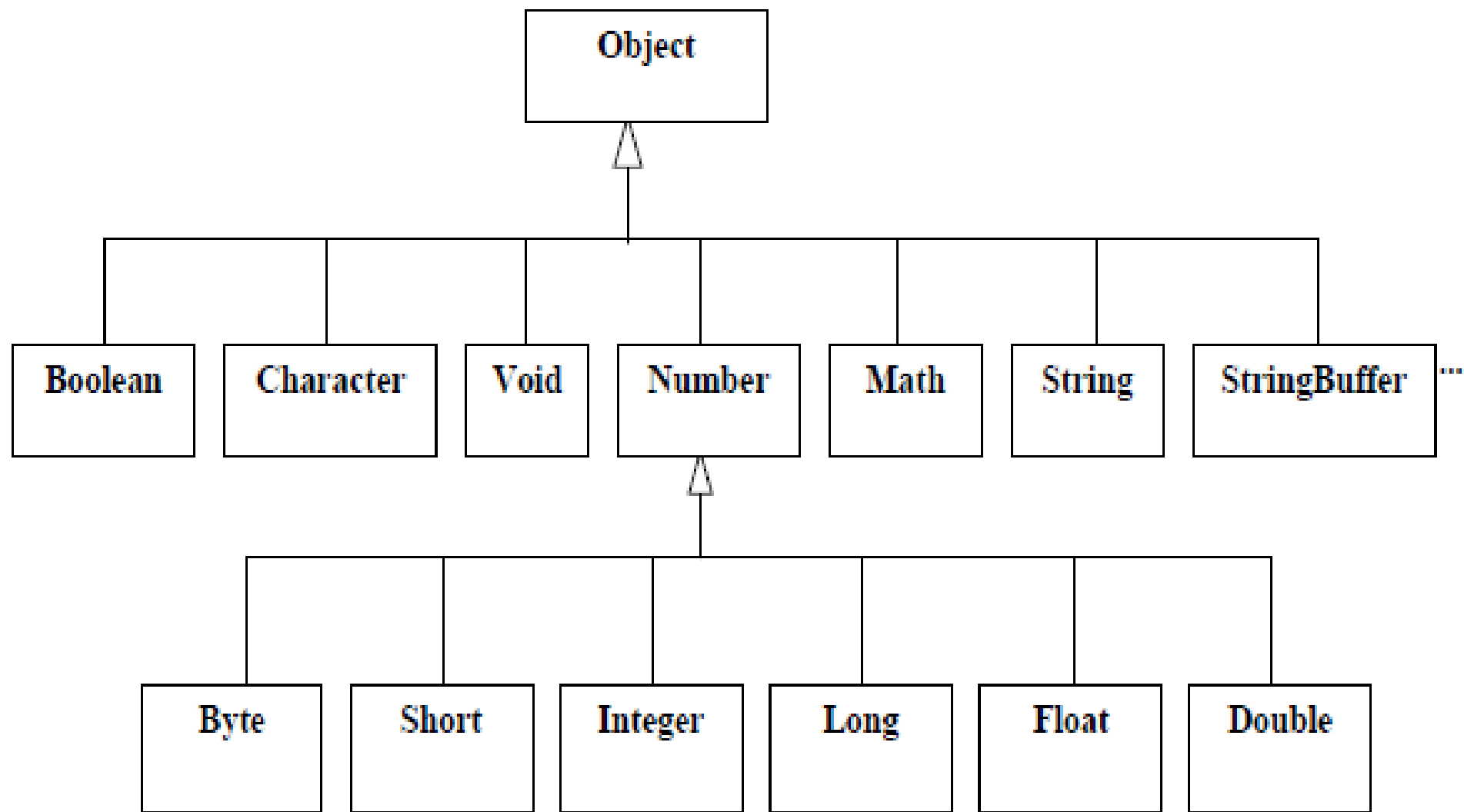
Một số lớp dữ liệu cơ bản trong Java

- ❑ Các kiểu dữ liệu nguyên thủy và lớp bao tương ứng trong Java:

Kiểu dữ liệu nguyên thủy	Lớp bao (Wrapper class)
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean



Một số lớp dữ liệu cơ bản trong Java





Một số lớp dữ liệu cơ bản trong Java

❑ Lưu ý:

❑ Boxing và Unboxing:

- Boxing: Quá trình tự động chuyển đổi một kiểu dữ liệu nguyên thủy sang đối tượng lớp bao tương ứng.
- Unboxing: Quá trình ngược lại, chuyển đổi từ đối tượng của lớp bao về kiểu dữ liệu nguyên thủy.



Một số lớp dữ liệu cơ bản trong Java

```
public class Main {  
    public static void main(String[] args) {  
        // Boxing: Từ int sang Integer  
        int a = 5;  
        Integer aWrapper = a;  
        // Unboxing: Từ Integer sang int  
        Integer bWrapper = new Integer(10);  
        int b = bWrapper;  
        System.out.println("Boxing: " + aWrapper);  
        System.out.println("Unboxing: " + b);  
    }  
}
```



Một số lớp dữ liệu cơ bản trong Java

□ Lưu ý:

- So sánh giữa kiểu nguyên thủy và lớp bao bọc:
 - Kiểu nguyên thủy: không có phương thức và không thể là *null*.
 - Lớp bao bọc: có thể có giá trị *null* và có các phương thức bổ sung như *compareTo()*, *equals()*, ...



Một số lớp dữ liệu cơ bản trong Java

- ❑ **Lưu ý:**
 - ❑ Tại sao cần dùng Wrapper Classes:
 - Để sử dụng các kiểu dữ liệu nguyên thủy trong các cấu trúc dữ liệu như ArrayList, HashMap, vốn yêu cầu các đối tượng.
 - Cung cấp các phương thức tiện ích để chuyển đổi và thao tác với dữ liệu.



Lớp Object

- ❑ Lớp **Object** là lớp gốc (root class) của tất cả các lớp trong Java.
- ❑ Mọi lớp trong Java đều mặc định kế thừa từ **Object**.
- ❑ Ngay cả khi không trực tiếp khai báo việc kế thừa từ **Object**, mỗi lớp đều mặc định kế thừa từ nó.



Lớp Object

Phương thức	Mô tả
String toString()	Trả về chuỗi mô tả đối tượng.
boolean equals(Object obj)	So sánh hai đối tượng xem có bằng nhau không.
int hashCode()	Trả về giá trị băm (hash code) của đối tượng.
Object clone()	Tạo một bản sao của đối tượng.
void finalize()	Được gọi bởi Garbage Collector trước khi đối tượng bị xóa.
Class<?> getClass()	Trả về đối tượng Class của đối tượng hiện tại.
void notify()	Đánh thức một luồng đang chờ trên đối tượng này.
void notifyAll()	Đánh thức tất cả các luồng đang chờ trên đối tượng này.
void wait()	Tạm dừng luồng cho đến khi được thông báo.
void wait(long timeout)	Tạm dừng luồng trong một khoảng thời gian cụ thể.
void wait(long timeout, int nanos)	Tạm dừng luồng trong khoảng thời gian cụ thể (gồm cả nanoseconds).



Lớp String

- ❑ Lớp String là một lớp bất biến (immutable) và rất phổ biến, được sử dụng để đại diện và thao tác với chuỗi ký tự.
- ❑ Bất biến (Immutable): Một đối tượng String không thể thay đổi sau khi được tạo. Nếu thay đổi chuỗi, một đối tượng String mới sẽ được tạo ra, và đối tượng cũ vẫn giữ nguyên.
- ❑ Chuỗi ký tự Unicode: Lớp String sử dụng mã hóa Unicode để lưu trữ chuỗi, cho phép biểu diễn ký tự từ nhiều ngôn ngữ khác nhau.



Lớp String

- ❑ Có hai cách phổ biến để tạo chuỗi trong Java:
 - ❑ Sử dụng chuỗi ký tự trực tiếp: `String s = "Hello";`
 - ❑ Sử dụng từ khóa `new`: `String s = new String("Hello");`
 - ❑ String hỗ trợ chuỗi trống (""), và chuỗi **null** (**null**). Chuỗi trống là chuỗi không có ký tự, trong khi **null** là khi chuỗi không được khởi tạo.



Lớp String

❑ Một số phương thức quan trọng của lớp String:

Phương thức	Mô tả	Phương thức	Mô tả
<code>int length()</code>	Trả về độ dài của chuỗi.	<code>boolean equalsIgnoreCase(String another)</code>	So sánh chuỗi với một chuỗi khác, bỏ qua phân biệt chữ hoa/chữ thường.
<code>char charAt(int index)</code>	Trả về ký tự tại vị trí chỉ định.	<code>int compareTo(String another)</code>	So sánh hai chuỗi theo từ điển. Trả về số nguyên: 0 nếu bằng, >0 hoặc <0.
<code>String substring(int beginIndex)</code>	Trả về chuỗi con bắt đầu từ vị trí beginIndex.	<code>boolean startsWith(String prefix)</code>	Kiểm tra chuỗi có bắt đầu bằng chuỗi con chỉ định không.
<code>String substring(int beginIndex, int endIndex)</code>	Trả về chuỗi con từ beginIndex đến endIndex - 1.	<code>boolean endsWith(String suffix)</code>	Kiểm tra chuỗi có kết thúc bằng chuỗi con chỉ định không.
<code>boolean equals(Object another)</code>	So sánh chuỗi với một đối tượng khác, trả về true nếu bằng nhau.	<code>String replace(char oldChar, char newChar)</code>	Thay thế tất cả các ký tự oldChar bằng newChar trong chuỗi.
<code>String concat(String str)</code>	Nối chuỗi hiện tại với chuỗi str.		



Lớp String

```
public class Main {  
    public static void main(String[] args) {  
        String s = "Hello, World!";  
        // Độ dài của chuỗi  
        System.out.println("Length: " + s.length()); // Kết quả: 13  
        // Trả về ký tự tại vị trí chỉ định  
        System.out.println("Char at index 1: " + s.charAt(1)); // Kết quả: 'e'  
        // Kiểm tra chuỗi có bắt đầu bằng "Hello" không  
        System.out.println("Starts with 'Hello': " + s.startsWith("Hello")); // Kết quả: true  
        // Chuyển chuỗi thành chữ hoa  
        System.out.println("Uppercase: " + s.toUpperCase()); // Kết quả: "HELLO, WORLD!"  
        // Tạo chuỗi con  
        System.out.println("Substring: " + s.substring(7, 12)); // Kết quả: "World"  
        // Thay thế ký tự  
        System.out.println("Replace 'l' with 'x': " + s.replace('l', 'x')); // Kết quả: "Hexxo, Worxd!"  
    }  
}
```



Lớp StringBuffer

- ❑ Lớp StringBuffer trong Java là một lớp được sử dụng để thao tác với các chuỗi ký tự giống như lớp String.
- ❑ Lớp StringBuffer cho phép thay đổi nội dung của đối tượng sau khi được tạo mà không cần tạo ra đối tượng mới.
- ❑ Lớp StringBuffer đồng bộ hóa các phương thức của nó, điều này làm cho nó an toàn khi sử dụng trong môi trường đa luồng.
- ❑ Hiệu suất của StringBuffer tốt hơn String trong trường hợp cần thực hiện nhiều phép toán thay đổi chuỗi



Lớp StringBuffer

- ❑ Khởi tạo: `StringBuffer sb = new StringBuffer("Hello");`
- ❑ Một số phương thức:

Phương thức	Mô tả
<code>append(String str)</code>	Thêm chuỗi str vào cuối đối tượng StringBuffer.
<code>insert(int offset, String str)</code>	Chèn chuỗi str vào vị trí chỉ định.
<code>replace(int start, int end, String str)</code>	Thay thế chuỗi từ vị trí start đến end bằng chuỗi str.
<code>delete(int start, int end)</code>	Xóa chuỗi từ vị trí start đến end.
<code>deleteCharAt(int index)</code>	Xóa ký tự tại vị trí chỉ định.
<code>reverse()</code>	Đảo ngược chuỗi trong StringBuffer.
<code>charAt(int index)</code>	Trả về ký tự tại vị trí chỉ định.
<code>length()</code>	Trả về độ dài của chuỗi.
<code>capacity()</code>	Trả về dung lượng hiện tại của StringBuffer.
<code>setCharAt(int index, char ch)</code>	Thay thế ký tự tại vị trí chỉ định bằng ký tự ch.
<code>substring(int start)</code>	Trả về chuỗi con bắt đầu từ vị trí start.
<code>substring(int start, int end)</code>	Trả về chuỗi con từ vị trí start đến end.



Lớp StringBuffer

```
public class Main {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer("Hello");  
        // Thêm chuỗi vào cuối  
        sb.append(" World");  
        System.out.println("After append: " + sb); // Kết quả: "Hello World"  
        // Chèn chuỗi tại vị trí chỉ định  
        sb.insert(5, ",");  
        System.out.println("After insert: " + sb); // Kết quả: "Hello, World"  
        // Thay thế chuỗi  
        sb.replace(6, 11, "Java");  
        System.out.println("After replace: " + sb); // Kết quả: "Hello, Java"  
        // Xóa một phần của chuỗi  
        sb.delete(5, 6);  
        System.out.println("After delete: " + sb); // Kết quả: "Hello Java"  
        // Đảo ngược chuỗi  
        sb.reverse();  
        System.out.println("After reverse: " + sb); // Kết quả: "avaJ olleH"  
    }  
}
```



Lớp Math

- ❑ Lớp *Math* trong Java cung cấp các phương thức toán học để thực hiện các phép tính cơ bản và nâng cao.
- ❑ Tất cả các phương thức trong lớp *Math* đều là **static** (tĩnh), do đó có thể sử dụng trực tiếp mà không cần tạo đối tượng của lớp *Math*.
- ❑ Lớp *Math* cung cấp các hằng số toán học như PI (π) và E (số Euler).



Lớp Math

❑ Một số phương thức:

Phương thức/Hằng số	Mô tả
double atan(double a)	Trả về giá trị arctan của a (ngược của tan, trả về giá trị theo radian).
long round(double a)	Làm tròn số a tới số nguyên gần nhất.
int max(int a, int b)	Trả về giá trị lớn hơn giữa a và b.
int min(int a, int b)	Trả về giá trị nhỏ hơn giữa a và b.
double ceil(double a)	Trả về giá trị nguyên lớn hơn hoặc bằng a.
double floor(double a)	Trả về giá trị nguyên nhỏ hơn hoặc bằng a.
double random()	Trả về số ngẫu nhiên giữa 0.0 và 1.0.
double toRadians(double angdeg)	Chuyển đổi góc từ độ sang radian.
double toDegrees(double angrad)	Chuyển đổi góc từ radian sang độ.
PI	Hằng số π (3.14159...).
E	Hằng số e (2.718...), cơ số của logarithm tự nhiên.

Phương thức/Hằng số	Mô tả
double abs(double a)	Trả về giá trị tuyệt đối của số a.
double sqrt(double a)	Trả về căn bậc hai của số a.
double pow(double a, double b)	Trả về giá trị a mũ b (a^b).
double exp(double a)	Trả về giá trị của e^a , với e là số Euler (2.718...).
double log(double a)	Trả về logarithm cơ số tự nhiên ($\log_e$) của a.
double log10(double a)	Trả về logarithm cơ số 10 của a.
double sin(double a)	Trả về giá trị sin của góc a (theo radian).
double cos(double a)	Trả về giá trị cos của góc a (theo radian).
double tan(double a)	Trả về giá trị tan của góc a (theo radian).
double asin(double a)	Trả về giá trị arcsin của a (ngược của sin, trả về giá trị theo radian).
double acos(double a)	Trả về giá trị arccos của a (ngược của cos, trả về giá trị theo radian).



BÀI 08: CÁC LỚP CƠ SỞ VÀ CÁC LỚP CẤU TRÚC DỮ LIỆU

- ❑ Cấu trúc mảng trong Java
- ❑ Một số lớp dữ liệu cơ bản trong Java
- ❑ **Các lớp cấu trúc dữ liệu**
- ❑ Các lớp tuyến tập (container classes)
- ❑ Bài tập



Lớp Arrays

- ❑ Lớp *Arrays* trong Java là một lớp tiện ích thuộc gói *java.util*.
- ❑ Cung cấp các phương thức tĩnh (**static**) để thao tác với các mảng (*arrays*) như sắp xếp, tìm kiếm, so sánh, sao chép, và chuyển đổi mảng thành chuỗi.
- ❑ Lớp *Arrays* chỉ hoạt động với mảng và không thể sử dụng với các đối tượng không phải là mảng.



Lớp Arrays

❑ Một số phương thức:

Phương thức	Mô tả
<code>static String toString(int[] a)</code>	Chuyển đổi mảng thành chuỗi.
<code>static void sort(int[] a)</code>	Sắp xếp mảng theo thứ tự tăng dần.
<code>static int binarySearch(int[] a, int key)</code>	Tìm kiếm một giá trị trong mảng đã được sắp xếp bằng thuật toán nhị phân.
<code>static int[] copyOf(int[] original, int newLength)</code>	Sao chép mảng original với độ dài mới newLength.
<code>static boolean equals(int[] a, int[] b)</code>	So sánh hai mảng xem chúng có bằng nhau không.
<code>static void fill(int[] a, int val)</code>	Gán tất cả các phần tử trong mảng a với giá trị val.
<code>static List<T> asList(T... a)</code>	Chuyển mảng thành danh sách (List) không thay đổi.
<code>static int[] copyOfRange(int[] original, int from, int to)</code>	Sao chép mảng từ chỉ số from đến to.
<code>static void parallelSort(int[] a)</code>	Sắp xếp mảng bằng thuật toán sắp xếp song song (tối ưu cho mảng lớn).
<code>static int mismatch(int[] a, int[] b)</code>	Trả về vị trí đầu tiên mà hai mảng khác nhau.
<code>static int hashCode(int[] a)</code>	Tính giá trị băm của mảng.



Lớp Arrays

```
import java.util.Arrays;
public class Main {
    public static void main(String[] args) {
        int[] array = {5, 3, 8, 1, 2};
        // Chuyển đổi mảng thành chuỗi
        System.out.println("Array: " + Arrays.toString(array)); // Kết quả: [5, 3, 8, 1, 2]
        // Sắp xếp mảng
        Arrays.sort(array);
        System.out.println("Sorted array: " + Arrays.toString(array)); // Kết quả: [1, 2, 3, 5, 8]
        // Tìm kiếm nhị phân
        int index = Arrays.binarySearch(array, 3);
        System.out.println("Index of 3: " + index); // Kết quả: 2
        // Sao chép mảng
        int[] newArray = Arrays.copyOf(array, 7);
        System.out.println("Copied array: " + Arrays.toString(newArray)); // Kết quả: [1, 2, 3, 5, 8, 0, 0]
        // Kiểm tra mảng bằng nhau
        int[] array2 = {1, 2, 3, 5, 8};
        boolean isEqual = Arrays.equals(array, array2);
        System.out.println("Arrays equal: " + isEqual); // Kết quả: true
        // Điền giá trị vào mảng
        Arrays.fill(newArray, 10);
        System.out.println("Filled array: " + Arrays.toString(newArray)); // Kết quả: [10, 10, 10, 10, 10, 10, 10]
        // Tìm vị trí khác nhau giữa hai mảng
        int mismatchIndex = Arrays.mismatch(array, array2);
        System.out.println("Mismatch index: " + mismatchIndex); // Kết quả: -1 (không khác nhau)
    }
}
```



Lớp *ArrayList*

- ❑ Lớp *ArrayList* trong Java thuộc gói *java.util*
- ❑ Khi thêm hoặc xóa phần tử, kích thước của *ArrayList* sẽ tự động thay đổi.
- ❑ *ArrayList* có thể lưu trữ giá trị **null**.



Lớp `ArrayList`

- ❑ Lớp `ArrayList` trong Java thuộc gói `java.util`
- ❑ Khi thêm hoặc xóa phần tử, kích thước của `ArrayList` sẽ tự động thay đổi.
- ❑ `ArrayList` có thể lưu trữ giá trị **null**.
- ❑ Khởi tạo:

```
import java.util.ArrayList;  
ArrayList<String> list = new ArrayList<>();
```



Lớp ArrayList

❑ Một số phương thức:

Phương thức	Mô tả
boolean add(E e)	Thêm phần tử e vào cuối danh sách.
void add(int index, E element)	Thêm phần tử vào vị trí chỉ định trong danh sách.
E get(int index)	Trả về phần tử tại chỉ số index.
E set(int index, E element)	Gán phần tử tại vị trí index bằng phần tử mới và trả về phần tử cũ.
E remove(int index)	Xóa phần tử tại chỉ số index và trả về phần tử đã bị xóa.
boolean remove(Object o)	Xóa phần tử đầu tiên có giá trị o.
boolean contains(Object o)	Kiểm tra xem danh sách có chứa phần tử o hay không.
int size()	Trả về số lượng phần tử trong danh sách.
void clear()	Xóa tất cả các phần tử trong danh sách.
boolean isEmpty()	Kiểm tra xem danh sách có rỗng không.
int indexOf(Object o)	Trả về chỉ số của phần tử đầu tiên có giá trị o.
int lastIndexOf(Object o)	Trả về chỉ số của phần tử cuối cùng có giá trị o.
Object[] toArray()	Chuyển ArrayList thành mảng.
List<E> subList(int fromIndex, int toIndex)	Trả về một phần của danh sách từ fromIndex đến toIndex.



Lớp ArrayList

```
import java.util.ArrayList;
public class Main {
    public static void main(String[] args) {
        // Khởi tạo ArrayList
        ArrayList<String> list = new ArrayList<>();
        // Thêm phần tử vào danh sách
        list.add("Apple");
        list.add("Banana");
        list.add("Orange");
        // In danh sách
        System.out.println("ArrayList: " + list); // Kết quả: [Apple, Banana, Orange]
        // Lấy phần tử theo chỉ số
        String fruit = list.get(1);
        System.out.println("Phần tử tại chỉ số 1: " + fruit); // Kết quả: Banana
        // Thay đổi giá trị phần tử tại vị trí 2
        list.set(2, "Grapes");
        System.out.println("Sau khi thay đổi: " + list); // Kết quả: [Apple, Banana, Grapes]
        // Xóa phần tử theo chỉ số
        list.remove(1);
        System.out.println("Sau khi xóa: " + list); // Kết quả: [Apple, Grapes]
        // Kích thước của danh sách
        int size = list.size();
        System.out.println("Kích thước: " + size); // Kết quả: 2
        // Kiểm tra xem danh sách có chứa phần tử "Apple" hay không
        boolean containsApple = list.contains("Apple");
        System.out.println("Có chứa Apple: " + containsApple); // Kết quả: true
        // Xóa tất cả các phần tử
        list.clear();
        System.out.println("Danh sách sau khi xóa: " + list); // Kết quả: []
    }
}
```



Lớp ArrayList

- ❑ Điểm khác biệt giữa *ArrayList* và mảng:
 - ❑ Kích thước của *ArrayList* có thể thay đổi linh hoạt, trong khi kích thước của mảng là cố định khi khởi tạo.
 - ❑ *ArrayList* có hiệu suất tốt khi thao tác truy cập phần tử, nhưng mảng có hiệu suất cao hơn khi xử lý một số lượng lớn các phần tử vì không có chi phí quản lý danh sách động.
 - ❑ *ArrayList* chỉ làm việc với các đối tượng (*class*), không thể lưu trữ các kiểu dữ liệu nguyên thủy trực tiếp như **int**, **double**. Tuy nhiên, Java hỗ trợ tính năng **autoboxing** để tự động chuyển các kiểu dữ liệu nguyên thủy thành đối tượng (như Integer thay cho **int**).



BÀI 08: CÁC LỚP CƠ SỞ VÀ CÁC LỚP CẤU TRÚC DỮ LIỆU

- ❑ Cấu trúc mảng trong Java
- ❑ Một số lớp dữ liệu cơ bản trong Java
- ❑ Các lớp cấu trúc dữ liệu
- ❑ **Các lớp tuyển tập (container classes)**
- ❑ Bài tập

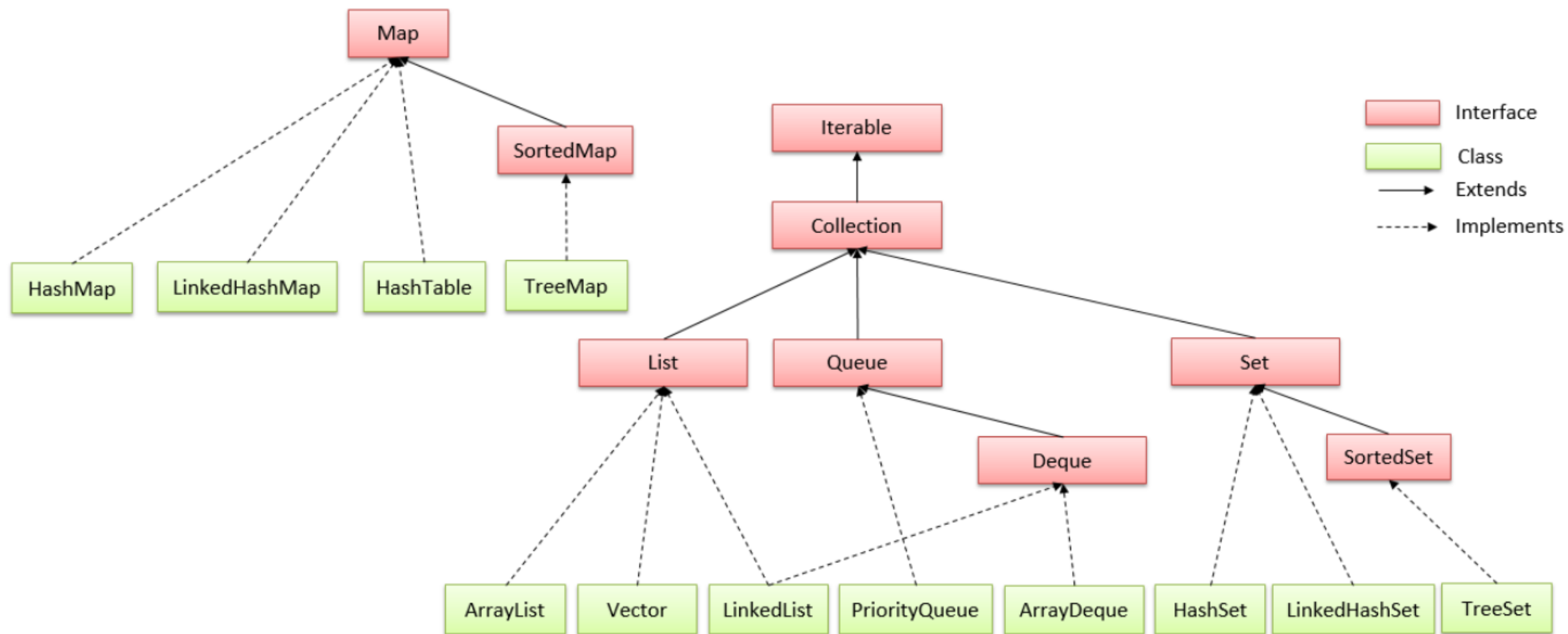


Các lớp tuyển tập (container classes)

- ❑ Các lớp tuyển tập thuộc gói `java.util` dùng để lưu trữ và quản lý các nhóm đối tượng.
- ❑ Được thiết kế để xử lý một loạt các cấu trúc dữ liệu khác nhau
- ❑ Các lớp tuyển tập được chia thành bốn nhóm chính dựa trên cách quản lý dữ liệu: **List**, **Set**, **Queue**, và **Map**.



Các lớp tuyển tập (container classes)





Iterator

- ❑ Iterator là một giao diện (interface) thuộc gói java.util được sử dụng để duyệt (traverse) qua các phần tử của một tuyển tập (collection)
- ❑ Iterator cung cấp một cách thức tiêu chuẩn để truy cập các phần tử của tuyển tập một cách tuần tự mà không cần quan tâm đến cách thức thực hiện bên trong của tuyển tập đó.



Iterator

❑ Các phương thức:

Phương thức	Mô tả
boolean hasNext()	Kiểm tra xem còn phần tử nào tiếp theo trong tuyển tập hay không.
E next()	Trả về phần tử tiếp theo trong tuyển tập và di chuyển con trỏ về phía trước.
void remove()	Xóa phần tử hiện tại khỏi tuyển tập (phần tử vừa được trả về bởi next()).

❑ Ví dụ:

```
import java.util.ArrayList;
import java.util.Iterator;
public class Main {
    public static void main(String[] args) {
        // Tạo một ArrayList
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Orange");
        // Sử dụng Iterator để duyệt qua danh sách
        Iterator<String> iterator = list.iterator();
        while (iterator.hasNext()) {
            String element = iterator.next();
            System.out.println(element); // In ra từng phần tử: Apple, Banana, Orange
        }
    }
}
```



Collection

- ❑ Collection trong Java là một giao diện gốc (root interface) trong `java.util` được sử dụng để lưu trữ và quản lý các nhóm đối tượng.
- ❑ Giao diện Collection cung cấp các phương thức cơ bản để thao tác với các tuyến tập như thêm, xóa, kiểm tra kích thước, kiểm tra rỗng, duyệt các phần tử, và chuyển đổi sang mảng.



Collection

❑ Các phương thức chính:

Phương thức	Mô tả
<code>boolean add(E e)</code>	Thêm phần tử e vào tuyển tập.
<code>boolean remove(Object o)</code>	Xóa phần tử o khỏi tuyển tập nếu nó tồn tại.
<code>boolean contains(Object o)</code>	Kiểm tra xem tuyển tập có chứa phần tử o không.
<code>int size()</code>	Trả về số lượng phần tử trong tuyển tập.
<code>boolean isEmpty()</code>	Kiểm tra xem tuyển tập có rỗng không.
<code>void clear()</code>	Xóa tất cả các phần tử trong tuyển tập.
<code>Iterator<E> iterator()</code>	Trả về một Iterator để duyệt qua các phần tử trong tuyển tập.
<code>boolean containsAll(Collection<?> c)</code>	Kiểm tra xem tuyển tập có chứa tất cả các phần tử của tuyển tập c.
<code>boolean addAll(Collection<? extends E> c)</code>	Thêm tất cả các phần tử của tuyển tập c vào tuyển tập hiện tại.
<code>boolean removeAll(Collection<?> c)</code>	Xóa tất cả các phần tử có trong tuyển tập c khỏi tuyển tập hiện tại.
<code>boolean retainAll(Collection<?> c)</code>	Giữ lại các phần tử có trong tuyển tập c và xóa phần tử còn lại.
<code>Object[] toArray()</code>	Chuyển đổi tuyển tập thành mảng các đối tượng.



Các lớp tuyển tập chính trong Java

Nhóm tuyển tập	Lớp chủ yếu	Mô tả
List	ArrayList, LinkedList, Vector, Stack	Lưu trữ các phần tử có thứ tự, cho phép phần tử trùng lặp.
Set	HashSet, LinkedHashSet, TreeSet	Lưu trữ các phần tử không trùng lặp, không đảm bảo thứ tự (ngoại trừ LinkedHashSet và TreeSet).
Queue	PriorityQueue, ArrayDeque, LinkedList	Quản lý các phần tử theo kiểu hàng đợi, thường được sử dụng theo nguyên tắc FIFO (first-in, first-out).
Map	HashMap, LinkedHashMap, TreeMap, Hashtable	Lưu trữ các cặp khóa-giá trị (key-value), mỗi khóa là duy nhất.



Collection

❑ Các phương thức chính:

Phương thức	Mô tả
<code>boolean add(E e)</code>	Thêm phần tử e vào tuyển tập.
<code>boolean remove(Object o)</code>	Xóa phần tử o khỏi tuyển tập nếu nó tồn tại.
<code>boolean contains(Object o)</code>	Kiểm tra xem tuyển tập có chứa phần tử o không.
<code>int size()</code>	Trả về số lượng phần tử trong tuyển tập.
<code>boolean isEmpty()</code>	Kiểm tra xem tuyển tập có rỗng không.
<code>void clear()</code>	Xóa tất cả các phần tử trong tuyển tập.
<code>Iterator<E> iterator()</code>	Trả về một Iterator để duyệt qua các phần tử trong tuyển tập.
<code>boolean containsAll(Collection<?> c)</code>	Kiểm tra xem tuyển tập có chứa tất cả các phần tử của tuyển tập c.
<code>boolean addAll(Collection<? extends E> c)</code>	Thêm tất cả các phần tử của tuyển tập c vào tuyển tập hiện tại.
<code>boolean removeAll(Collection<?> c)</code>	Xóa tất cả các phần tử có trong tuyển tập c khỏi tuyển tập hiện tại.
<code>boolean retainAll(Collection<?> c)</code>	Giữ lại các phần tử có trong tuyển tập c và xóa phần tử còn lại.
<code>Object[] toArray()</code>	Chuyển đổi tuyển tập thành mảng các đối tượng.



Bài tập

Tạo một chương trình Java sử dụng ArrayList để quản lý danh sách học sinh. Chương trình có các chức năng:

- ❑ Thêm học sinh mới vào danh sách.
- ❑ Xóa học sinh khỏi danh sách theo tên.
- ❑ Hiển thị tất cả học sinh hiện có trong danh sách.
- ❑ Kiểm tra xem một học sinh có tồn tại trong danh sách không.



BÀI 08: CÁC LỚP CƠ SỞ VÀ CÁC LỚP CẤU TRÚC DỮ LIỆU

- ❑ Cấu trúc mảng trong Java
- ❑ Một số lớp dữ liệu cơ bản trong Java
- ❑ Các lớp cấu trúc dữ liệu
- ❑ Các lớp tuyến tập (container classes)
- ❑ Bài tập



Sinh viên tự học

- ☐ Làm bài tập trắc nghiệm trên hệ thống LMS
- ☐ Bài tập kỹ năng (nếu có)



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM